Pattern matching e ricorsione strutturale

L04 · Programmazione funzionale

Mauro Leoncini

Obiettivi

- Costruttori e decomposizione
- La lista come **tipo ricorsivo**, non come struttura primitiva
- La ricorsione che segue la forma del dato
- L'esaustività del *pattern matching* come **strumento di verifica**

Da dove viene una lista

Non ci sarebbe bisogno che il linguaggio ve la **regali** (cioè la metta a disposizione come tipo predefinito).

Una lista di `'a` è una di due cose che possiamo benissimo definire, anche usando i termini che più ci **garbano**:

```
type 'a lista =
  | Vuota
  | Cons of 'a * 'a lista
```

`Vuota` e `Cons` sono i due **costruttori** che abbiamo scelto.

Benché la definizione faccia riferimento a se stessa (una lista contiene una lista), essa **non è circolare** perché "ricorre" solo su un ramo.

[] e :: hanno la stessa forma

```
Cons (1, Cons (2, Cons (3, Vuota)))     (* int lista *)
(::) (1, (::) (2, (::) (3, [])))        (* int list  *)
1 :: 2 :: 3 :: []                       (* int list  *)
[1; 2; 3]                               (* int list  *)
```

Le ultime tre scritture indicano **la stessa cosa** in tre modi diversi. La prima ha la **stessa forma**, costruttore per costruttore, ma OCaml la tratta come avente un altro tipo.

Le liste di OCaml non sono un tipo speciale del linguaggio: sono un tipo **definito per casi**, proprio come `lista`, con una sintassi (un pò) più comoda.

Nota Le parentesi intorno a `::` servono ad evitare un problema di parsing. Infatti `::` è un costruttore, ma possiede una sintassi infissa speciale. Senza parentesi, il parser lo riconosce soltanto nella forma `espressione :: espressione`

Stessa forma, tipi diversi

Provate a confrontare la prima con le altre:

```
Cons (1, Cons (2, Cons (3, Vuota))) = [1; 2; 3];;
Error: This variant expression is expected to have type int lista
       ...
```

OCaml distingue i tipi definiti per casi **rispetto al nome**, non rispetto alla forma. Ed è una garanzia, non un fastidio:

```
type euro    = Euro of float
type dollaro = Dollaro of float
```

Costruttori di valori e di tipi

Un **costruttore di valori** descrive una delle forme che può assumere un valore di un tipo variante.

```
type forma =
  | Punto
  | Cerchio of float
  | Rettangolo of float * float
```

```
Punto       : forma
Cerchio     : float -> forma
Rettangolo  : float * float -> forma
```

La keyword `of` introduce il tipo del **contenuto trasportato**. Un costruttore serve sia per costruire valori sia per riconoscerli e decomporli mediante *pattern matching*; il suo nome inizia normalmente con una maiuscola.

Costruttori di tipi

Un **costruttore di tipi** prende invece uno o più tipi come argomenti e produce un nuovo tipo:

```
int list             string list
int option           string option
(string, int) result
```

`option` costruisce tipi, mentre `Some` e `None` costruiscono valori:

```
int option       (* tipo *)
Some 3           (* valore di tipo int option *)
None             (* valore di tipo 'a option *)
```

Pattern matching: the `OCaml`'s way!

Un classico attraversamento ricorsivo di una `int list` per sommare i suoi elementi

```
let rec somma l = if l = []
   then 0
   else List.hd l + somma (List.tl l)
```

- Possiamo ripetere il procedimento con una `lista` (come l'abbiamo definita noi)?
- Come li "prendiamo" gli equivalenti di head e tail delle `list` predefinite?
- La risposta è il *pattern matching*

```
let rec somma l =
    match l with
      | [] -> 0
      | x :: tl -> x + somma tl
```

Cosa fa `match`

Il `match` svolge due funzioni cruciali:

- come prima cosa **distingue** quale costruttore è stato usato
- in secondo luogo **lega** nomi (arbitrari, scelti dal programmatore) alle componenti che quel costruttore conteneva
- *match* non ha nulla a che vedere con il costrutto `switch` presente in altri linguaggi.

Nel caso dell'esempio, `x` e `tl` (che esistono solo nel ramo in cui hanno senso) vengono legati alla testa e alla coda della lista.

La ricetta generale

> Si definisce un caso **per ogni costruttore** del tipo. Nel caso ricorsivo, richiama la funzione su una **porzione più piccola** della struttura. Non esiste un **caso default**: tutte le alternative vanno specificate.

Non è uno stile di programmazione: è la **struttura del tipo** che detta la struttura del pattern matching (e quindi della funzione). Se il tipo ha due costruttori, la funzione ha due casi.

Match applicato alle "nostre" liste

Con il pattern matching non abbiamo bisogno di **selettori** specifici per le varie parti della struttura

```
let rec somma l =
    match l with
      | Vuota -> 0
      | Cons (x, xl) -> x + somma xl
```

```
somma Cons (1, Cons (2, Cons (3, Vuota)))
-> 6
```

`length`, derivata dal tipo

```
let rec lunghezza l =
  match l with
  | [] -> 0
  | _ :: resto -> 1 + lunghezza resto
```

Il caso base è il costruttore che non ricorre. Il caso ricorsivo è quello che ricorre — e la chiamata avviene su `resto`, che è più corto.

Non abbiamo deciso noi la forma. L'ha decisa `type`.

Implementazione di `append` con pattern matching

Riscriviamo la funzione `append` che già abbiamo visto con l'uso classico dell'espressione `if ... then ... else`

```
let rec append l1 l2 =
  match l1 with
  | [] -> l2
  | x :: resto -> x :: append resto l2
```

Se dimentichiamo un caso

```
let rec lung l =
  match l with
  | Cons (_, resto) -> 1 + lung resto
```

```
Warning 8 [partial-match]: this pattern-matching
  is not exhaustive.
  Here is an example of a case that is not matched: Vuota
```

Il compilatore non dice «hai sbagliato». Ci consegna **un controesempio**.

Questo è il primo verificatore del corso

- il tipo (L03) dice che cosa la funzione può fare
- l'esaustività dice quali casi abbiamo (eventualmente) dimenticato, **con l'esempio**

In L09 uno strumento farà la stessa cosa su proprietà che scriviamo noi, e ci restituirà controesempi minimizzati. È lo stesso mestiere.

Dobbiamo trattare l'avviso come un errore. Se lo silenziamo perdiamo l'unica rete gratuita che ha.

I limiti della rete

Con una guardia il match resta esaustivo ai nostri occhi, non al compilatore:

```
match l with
| x :: _ when x > 0 -> "positivo"
| x :: _ when x <= 0 -> "non positivo"
| [] -> "vuota"
```

Il compilatore **non valuta le guardie**: non sa che i due casi coprono tutte le possibilità, e segnala comunque.

Ogni strumento di verifica ha un confine, e conviene sapere dov'è invece di scoprirlo.

`rev`, e un debito

```
let rec rev l =
  match l with
  | [] -> []
  | x :: resto -> append (rev resto) [x]
```

Corretta, e segue la ricetta. Ma `append` percorre ogni volta tutto ciò che ha già girato.

Provatela su una lista di centomila elementi. Il conto lo paghiamo in L07: qui ci interessa che sia **giusta**, non che sia veloce.

La forma è il punto

La forma del **tipo** ha dettato la forma della **funzione**.

In L08 vedrete che detta anche la forma della **dimostrazione**: per provare una proprietà di `lunghezza` si prova sul caso `[]` e la si eredita nel caso `x :: resto`.

Un caso per costruttore, tre volte: nel tipo, nel codice, nella prova.

Un'ultima cosa, per dopo

Il pattern matching decompone **a senso unico**: il dato è concreto, il pattern contiene i buchi.

```
match [1; 2] with x :: resto -> ...
```

In L14 incontreremo l'unificazione, dove i buchi possono stare **da entrambe le parti**. È il pattern matching senza la restrizione, ed è ciò che rende Prolog reversibile.

Che cosa portate a casa

- Una lista è un tipo somma ricorsivo, non un dono del linguaggio
- Il `match` distingue **e** lega, nello stesso gesto
- Un caso per costruttore; si ricorre sulla parte più piccola
- L'avviso di non esaustività è un controesempio, non un fastidio
- Ogni verificatore ha un confine: le guardie sono il confine di questo

Da fare prima di L05

In `codice/`:

1. `lista.ml` — reimplementate `lunghezza`, `append` e `rev` sul tipo `'a lista` definito da voi, senza usare le liste di OCaml
2. `esaustivita.ml` — istruzioni dentro il file: togliete un caso e leggete che cosa vi risponde il compilatore
3. `esercizi.ml` — `ultimo`, `pari`, `massimo`: tre funzioni, sempre con la ricetta

Chi finisce presto: `append` è associativa? Provate a convincervene. In L08 la dimostrerete.